Agent Hub — Project Structure & UI/UX Technical Specifications

> **Purpose**: Scalable folder structure for multi-person GitHub collaboration + complete UI/UX technical specs for fast, consistent, professional frontend development.
> **Stack**: React 19 (JS) · Vite · Tailwind CSS 4 · shadcn/ui · Zustand · Firebase Auth · Firestore · WebSocket
> **Theme**: Dark-first design with light mode support

Table of Contents

1. Repository Structure
2. Git Collaboration Strategy
3. Frontend Architecture
4. Design System & Theming
5. UI Component Library
6. Page-by-Page UI/UX Specs
7. API Layer & Security
8. WebSocket Protocol
9. State Management
10. Accessibility & Responsive Design
11. Performance Guidelines
12. Developer Workflow

---

1. Repository Structure

Monorepo Layout

```
agent-hub/
│
├── .github/                          # GitHub collaboration config
│   ├── workflows/
│   │   ├── ci-backend.yml            # Python lint + test on PR
│   │   ├── ci-frontend.yml           # JS lint + build on PR
│   │   ├── deploy-staging.yml        # Auto-deploy to staging on merge to develop
│   │   └── deploy-prod.yml           # Deploy to prod on merge to main
│   ├── PULL_REQUEST_TEMPLATE.md      # Standardized PR template
│   ├── ISSUE_TEMPLATE/
│   │   ├── bug_report.yml
│   │   ├── feature_request.yml
│   │   └── task.yml
│   └── CODEOWNERS                    # Auto-assign reviewers per folder
│
├── backend/                          # Python + FastAPI + ADK
│   ├── pyproject.toml                # uv project config + deps
│   ├── uv.lock                       # Deterministic lockfile
│   ├── Dockerfile                    # Cloud Run container
│   ├── .env.example                  # Template (never commit .env)
│   ├── ruff.toml                     # Linting config
│   │
│   ├── app/                          # Application root
│   │   ├── __init__.py
│   │   ├── main.py                   # FastAPI app factory + lifespan
│   │   ├── config.py                 # Settings (env vars, model config)
│   │   │
│   │   ├── api/                      # HTTP + WebSocket endpoints
│   │   │   ├── __init__.py
│   │   │   ├── router.py             # FastAPI router aggregator
│   │   │   ├── ws_live.py            # WebSocket /ws/live/{session_id} — ADK bidi streaming
│   │   │   ├── ws_events.py          # WebSocket /ws/events/{session_id} — dashboard events
│   │   │   ├── auth.py               # POST /auth/verify — Firebase token verification
│   │   │   ├── personas.py           # CRUD /personas — agent persona management
│   │   │   ├── sessions.py           # GET/DELETE /sessions — session history
│   │   │   ├── mcp.py                # GET/POST /mcp — plugin store management
│   │   │   ├── clients.py            # GET /clients — connected device status
│   │   │   └── health.py             # GET /health — Cloud Run healthcheck
│   │   │
│   │   ├── agents/                   # ADK agent definitions
│   │   │   ├── __init__.py
│   │   │   ├── root_agent.py         # Root router agent (LlmAgent)
│   │   │   ├── personas.py           # Persona sub-agents (Assistant, Coder, Researcher, Analyst, Creative)
│   │   │   ├── task_architect.py     # Meta-orchestrator (CustomAgent)
│   │   │   └── agent_factory.py      # Dynamic agent creation from persona config
│   │   │
│   │   ├── tools/                    # ADK custom tool functions
│   │   │   ├── __init__.py
│   │   │   ├── cross_client.py       # cross_client_action(), list_connected_clients()
│   │   │   ├── image_gen.py          # generate_image() — Nano Banana interleaved output
│   │   │   ├── desktop_tools.py      # capture_screen(), click_at(), type_text()
│   │   │   ├── code_exec.py          # execute_code() — E2B sandbox
│   │   │   └── search.py             # google_search grounding
│   │   │
│   │   ├── services/                 # Business logic (stateless)
│   │   │   ├── __init__.py
│   │   │   ├── client_registry.py    # In-memory client tracking
│   │   │   ├── mcp_manager.py        # Dynamic McpToolset instantiation
│   │   │   ├── e2b_service.py        # E2B sandbox lifecycle
│   │   │   ├── persona_service.py    # Firestore CRUD for personas
│   │   │   ├── session_service.py    # ADK session management
│   │   │   └── storage_service.py    # GCS image/file storage
│   │   │
│   │   ├── middleware/               # FastAPI middleware
│   │   │   ├── __init__.py
│   │   │   ├── auth_middleware.py    # Firebase JWT verification
│   │   │   ├── cors.py               # CORS config
│   │   │   └── rate_limit.py         # Request rate limiting
│   │   │
│   │   ├── models/                   # Pydantic schemas
│   │   │   ├── __init__.py
```

```
│   │   │   ├── persona.py             # PersonaCreate, PersonaResponse
│   │   │   ├── session.py             # SessionInfo, SessionList
│   │   │   ├── mcp.py                 # MCPConfig, MCPToggle
│   │   │   ├── client.py             # ClientInfo, ClientStatus
│   │   │   └── ws_messages.py         # WebSocket message schemas
│   │   │
│   │   └── utils/                     # Shared utilities
│   │       ├── __init__.py
│   │       ├── logging.py             # Structured logging setup
│   │       └── errors.py              # Custom exception classes
│   │
│   └── tests/                         # Pytest tests
│       ├── conftest.py                # Fixtures (mock Firebase, mock ADK)
│       ├── test_api/
│       ├── test_agents/
│       ├── test_services/
│       └── test_tools/
│
├── dashboard/                         # React 19 (JS) + Vite
│   ├── package.json
│   ├── vite.config.js
│   ├── jsconfig.json                  # Path aliases (@/components, @/hooks, etc.)
│   ├── tailwind.config.js             # Tailwind theme + dark mode
│   ├── .eslintrc.cjs                  # ESLint config
│   ├── .prettierrc                    # Prettier config
│   ├── index.html
│   │
│   ├── public/
│   │   ├── favicon.svg
│   │   ├── manifest.json              # PWA manifest
│   │   ├── sw.js                      # Service worker (offline PWA)
│   │   └── icons/                     # PWA icons (192x192, 512x512)
│   │
│   └── src/
│       ├── main.jsx                   # React root + providers
│       ├── App.jsx                    # Router + layout shell
│       │
│       ├── components/                # Reusable UI components
│       │   ├── ui/                    # shadcn/ui primitives (auto-generated)
│       │   │   ├── button.jsx
│       │   │   ├── input.jsx
│       │   │   ├── dialog.jsx
│       │   │   ├── dropdown-menu.jsx
│       │   │   ├── toast.jsx          # Sonner toast wrapper
│       │   │   ├── tooltip.jsx
│       │   │   ├── badge.jsx
│       │   │   ├── card.jsx
│       │   │   ├── avatar.jsx
│       │   │   ├── tabs.jsx
│       │   │   ├── switch.jsx
│       │   │   ├── slider.jsx
│       │   │   ├── skeleton.jsx
│       │   │   ├── scroll-area.jsx
│       │   │   ├── separator.jsx
│       │   │   ├── sheet.jsx          # Slide-over panel
│       │   │   ├── select.jsx
│       │   │   ├── popover.jsx
│       │   │   ├── command.jsx        # Command palette (⌘K)
│       │   │   └── sonner.jsx         # Toast provider
│       │   │
│       │   ├── layout/                # Layout components
│       │   │   ├── AppShell.jsx       # Main layout: sidebar + content
│       │   │   ├── Sidebar.jsx        # Navigation sidebar
│       │   │   ├── TopBar.jsx         # Header bar with user menu
│       │   │   ├── MobileNav.jsx      # Bottom nav for mobile
│       │   │   └── ThemeToggle.jsx    # Dark/light mode switcher
│       │   │
│       │   ├── chat/                  # Voice chat components
│       │   │   ├── ChatPanel.jsx      # Main chat container
│       │   │   ├── MessageBubble.jsx  # Single message (text/audio/genui)
│       │   │   ├── VoiceOrb.jsx       # Central voice activation orb
│       │   │   ├── Waveform.jsx       # Real-time audio waveform
│       │   │   ├── TranscriptLine.jsx # Single transcription line
│       │   │   ├── TypingIndicator.jsx # Agent thinking/processing
│       │   │   └── ChatInput.jsx      # Text input + mic toggle
│       │   │
│       │   ├── genui/                 # Generative UI components
│       │   │   ├── GenUIRenderer.jsx  # Routes type → component
│       │   │   ├── DynamicChart.jsx   # Line/bar/pie charts (Recharts)
│       │   │   ├── DataTable.jsx      # Sortable, filterable tables
```

```
│       │   │   ├── InfoCard.jsx        # Summary card
│       │   │   ├── CodeBlock.jsx       # Syntax-highlighted code
│       │   │   ├── ImageGallery.jsx    # Image grid + lightbox
│       │   │   ├── TimelineView.jsx    # Step timeline (TaskArchitect)
│       │   │   ├── MarkdownRenderer.jsx # Rich markdown
│       │   │   ├── DiffViewer.jsx      # Code diff
│       │   │   ├── WeatherWidget.jsx   # Weather card
│       │   │   └── MapView.jsx         # Embedded map
│       │   │
│       │   ├── persona/               # Persona management
│       │   │   ├── PersonaCard.jsx     # Single persona display
│       │   │   ├── PersonaList.jsx     # Grid of personas
│       │   │   ├── PersonaEditor.jsx   # Create/edit persona modal
│       │   │   └── VoicePreview.jsx    # Audio preview of persona voice
│       │   │
│       │   ├── mcp/                    # MCP plugin store
│       │   │   ├── MCPStoreGrid.jsx    # Plugin cards grid
│       │   │   ├── MCPCard.jsx         # Single plugin card
│       │   │   ├── MCPDetail.jsx       # Plugin detail sheet
│       │   │   ├── MCPCategoryNav.jsx  # Category filter tabs
│       │   │   └── MCPToggle.jsx       # Enable/disable toggle
│       │   │
│       │   ├── clients/               # Connected devices
│       │   │   ├── ClientStatusBar.jsx # Sticky bar showing connected clients
│       │   │   ├── ClientCard.jsx      # Single client device card
│       │   │   └── ClientList.jsx      # All connected devices
│       │   │
│       │   ├── session/               # Session history
│       │   │   ├── SessionList.jsx     # Sidebar session list
│       │   │   ├── SessionItem.jsx     # Single session entry
│       │   │   └── SessionSearch.jsx   # Search past sessions
│       │   │
│       │   ├── sandbox/               # E2B sandbox
│       │   │   ├── SandboxConsole.jsx  # Terminal-like output
│       │   │   ├── CodeEditor.jsx      # Simple code input
│       │   │   └── FileExplorer.jsx    # Sandbox file tree
│       │   │
│       │   ├── auth/                  # Authentication
│       │   │   ├── LoginPage.jsx       # Google sign-in page
│       │   │   ├── AuthGuard.jsx       # Protected route wrapper
│       │   │   └── UserMenu.jsx        # Avatar + dropdown
│       │   │
│       │   └── shared/                # Cross-cutting components
│       │       ├── LoadingSpinner.jsx  # Consistent spinner
│       │       ├── ErrorBoundary.jsx   # React error boundary
│       │       ├── EmptyState.jsx      # Empty list placeholder
│       │       ├── ConfirmDialog.jsx   # Destructive action confirmation
│       │       ├── StatusDot.jsx       # Online/offline indicator
│       │       ├── CopyButton.jsx      # Copy-to-clipboard button
│       │       └── KeyboardShortcut.jsx # Key combo display
│       │
│       ├── pages/                     # Route-level pages
│       │   ├── DashboardPage.jsx       # Main chat + GenUI workspace
│       │   ├── PersonasPage.jsx        # Persona management
│       │   ├── MCPStorePage.jsx        # Plugin store
│       │   ├── SessionsPage.jsx        # Session history
│       │   ├── SettingsPage.jsx        # User settings + preferences
│       │   ├── ClientsPage.jsx         # Connected devices
│       │   └── NotFoundPage.jsx        # 404
│       │
│       ├── stores/                    # Zustand state stores
│       │   ├── authStore.js            # User auth state
│       │   ├── chatStore.js            # Messages, transcription, voice state
│       │   ├── personaStore.js         # Active persona, persona list
│       │   ├── mcpStore.js             # Installed MCPs, store catalog
│       │   ├── clientStore.js          # Connected devices
│       │   ├── sessionStore.js         # Session list, active session
│       │   ├── themeStore.js           # Dark/light mode
│       │   └── uiStore.js             # Sidebar open, modals, command palette
│       │
│       ├── hooks/                     # Custom React hooks
│       │   ├── useWebSocket.js         # WebSocket connection lifecycle
│       │   ├── useAudioCapture.js      # Mic → PCM 16kHz → WebSocket
│       │   ├── useAudioPlayback.js     # WebSocket → PCM 24kHz → speakers
│       │   ├── useAuth.js              # Firebase auth hook
│       │   ├── useFirestore.js         # Firestore read/write
│       │   ├── useKeyboard.js          # Keyboard shortcut handler
│       │   └── useMediaQuery.js        # Responsive breakpoint detection
│       │
│       ├── lib/                       # Utilities & config
```

```
│       │   ├── firebase.js            # Firebase app init + auth + Firestore
│       │   ├── api.js                 # HTTP API client wrapper (fetch + token injection)
│       │   ├── ws.js                  # WebSocket client with reconnect + auth
│       │   ├── audio.js               # AudioWorklet processors
│       │   ├── constants.js           # App-wide constants
│       │   ├── cn.js                  # clsx + tailwind-merge utility
│       │   └── formatters.js          # Date, file size, duration formatters
│       │
│       └── styles/
│           └── globals.css            # Tailwind base + theme CSS variables
│
├── desktop-client/                    # Python tray app
│   ├── pyproject.toml
│   ├── uv.lock
│   ├── src/
│   │   ├── main.py                    # Entry point + tray icon
│   │   ├── ws_client.py               # WebSocket connection to hub
│   │   ├── screen.py                  # Screenshot capture
│   │   ├── actions.py                 # Mouse/keyboard/window actions
│   │   ├── files.py                   # File system operations
│   │   └── config.py                  # Settings (server URL, auth token)
│   └── tests/
│       └── test_actions.py
│
├── chrome-extension/                  # Manifest V3
│   ├── manifest.json
│   ├── background.js                  # Service worker: WebSocket + message routing
│   ├── content.js                     # Page interaction: DOM extraction, form fill
│   ├── popup/
│   │   ├── popup.html                 # Extension popup UI
│   │   ├── popup.js                   # Popup logic
│   │   └── popup.css                  # Popup styles (Tailwind CDN or inline)
│   ├── offscreen/
│   │   ├── offscreen.html             # Offscreen document for audio capture
│   │   └── offscreen.js              # Audio capture in service worker context
│   └── icons/
│       ├── icon-16.png
│       ├── icon-48.png
│       └── icon-128.png
│
├── deploy/                            # Infrastructure as Code
│   ├── terraform/
│   │   ├── main.tf                    # Cloud Run + Firestore + GCS
│   │   ├── variables.tf
│   │   └── outputs.tf
│   ├── docker-compose.yml             # Local dev (backend + frontend)
│   └── scripts/
│       ├── deploy.sh                  # One-click deploy
│       ├── setup-env.sh               # Initial env setup
│       └── seed-data.sh               # Seed Firestore with default personas + MCPs
│
├── docs/                              # Documentation
│   ├── architecture.png               # System architecture diagram
│   ├── api-reference.md               # REST + WebSocket API docs
│   ├── setup-guide.md                 # Getting started for new developers
│   ├── contributing.md                # Code style, PR process, branch naming
│   └── demo-script.md                 # 4-min demo video script
│
├── .gitignore
├── .editorconfig                      # Consistent indent/encoding across editors
├── LICENSE
└── README.md                          # Project overview + quick start
```

CODEOWNERS (Auto-Assign Reviewers)

```
# .github/CODEOWNERS
# Backend team
/backend/                       @backend-lead
/backend/app/agents/            @ai-lead
/backend/app/tools/             @ai-lead

# Frontend team
/dashboard/                     @frontend-lead
/dashboard/src/components/ui/   @frontend-lead
/dashboard/src/components/genui/ @frontend-lead @ai-lead

# Infra
/deploy/                        @devops-lead
/.github/workflows/             @devops-lead

# Clients
/desktop-client/                @backend-lead
/chrome-extension/              @frontend-lead
```

---

2. Git Collaboration Strategy

Branch Model

```
main                    ← Production (deploy-prod.yml triggers)
  └── develop           ← Integration branch (deploy-staging.yml triggers)
      ├── feat/chat-ui           ← Feature branches
      ├── feat/persona-api
      ├── feat/mcp-store
      ├── fix/ws-reconnect
      └── chore/ci-pipeline
```

Branch Naming Convention

Prefix	Use	Example
`feat/`	New feature	`feat/voice-orb-animation`
`fix/`	Bug fix	`fix/audio-playback-glitch`
`chore/`	Config, CI, deps	`chore/update-adk-version`
`refactor/`	Code restructure	`refactor/split-chat-store`
`docs/`	Documentation	`docs/api-reference`

PR Rules

- All PRs target `develop` (not `main`)
- Require 1 approval from CODEOWNER
- CI must pass (lint + build)
- Squash merge only (clean history)
- PR title follows conventional commits: `feat: add persona voice preview`

Parallel Workstreams (Multi-Person)

Person	Folder Ownership	Can Work In Parallel?
Dev A — Backend/AI	`backend/app/agents/`, `backend/app/tools/`, `backend/app/services/`	✅ No conflict with frontend
Dev B — Frontend UI	`dashboard/src/components/`, `dashboard/src/pages/`, `dashboard/src/styles/`	✅ No conflict with backend
Dev C — Frontend Logic	`dashboard/src/stores/`, `dashboard/src/hooks/`, `dashboard/src/lib/`	✅ Minimal conflict with Dev B
Dev D — API/Infra	`backend/app/api/`, `deploy/`, `.github/workflows/`	✅ Interfaces defined first
Dev E — Clients	`desktop-client/`, `chrome-extension/`	✅ Fully isolated

Conflict-Free Contract

Frontend and backend teams agree on:

1. **WebSocket message schema** (defined in `backend/app/models/ws_messages.py` — single source of truth)
2. **REST API contracts** (auto-generated OpenAPI from FastAPI)
3. **Firestore document schemas** (documented in `docs/api-reference.md`)

Frontend mocks WebSocket events with a local JSON file during development. Backend team publishes message schema first.

---

3. Frontend Architecture

Routing

```
// App.jsx — React Router v7
import { BrowserRouter, Routes, Route } from 'react-router';

<BrowserRouter>
  <Routes>
    <Route element={<AuthGuard />}>
      <Route element={<AppShell />}>
        <Route index          element={<DashboardPage />} />
        <Route path="personas" element={<PersonasPage />} />
        <Route path="plugins"  element={<MCPStorePage />} />
        <Route path="sessions" element={<SessionsPage />} />
        <Route path="clients"  element={<ClientsPage />} />
        <Route path="settings" element={<SettingsPage />} />
      </Route>
    </Route>
    <Route path="login" element={<LoginPage />} />
    <Route path="*"     element={<NotFoundPage />} />
  </Routes>
</BrowserRouter>
```

Path Aliases (jsconfig.json)

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"]
    }
  }
}
```

Usage: `import { Button } from '@/components/ui/button'`

Provider Stack

```
// main.jsx
import { Toaster } from '@/components/ui/sonner';
import { ThemeProvider } from '@/components/layout/ThemeProvider';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ThemeProvider>
      <BrowserRouter>
        <App />
      </BrowserRouter>
      <Toaster richColors position="bottom-right" />
    </ThemeProvider>
  </StrictMode>
);
```

---

4. Design System & Theming

Color System (CSS Variables)

Dark-first design. All colors use CSS custom properties so shadcn/ui, Tailwind, and custom components share the same palette.

```
/* globals.css */
@import "tailwindcss";

@custom-variant dark (&:is(.dark *));

:root {
  /* Light mode */
  --background: oklch(0.98 0 0);                /* near-white */
  --foreground: oklch(0.15 0 0);                 /* near-black */
  --card: oklch(1 0 0);                          /* white cards */
  --card-foreground: oklch(0.15 0 0);
  --popover: oklch(1 0 0);
  --popover-foreground: oklch(0.15 0 0);
  --primary: oklch(0.55 0.2 260);                /* blue-violet brand */
  --primary-foreground: oklch(0.98 0 0);
  --secondary: oklch(0.93 0.01 260);             /* light tinted bg */
  --secondary-foreground: oklch(0.25 0 0);
  --muted: oklch(0.94 0.005 260);
  --muted-foreground: oklch(0.5 0 0);
  --accent: oklch(0.93 0.01 260);
  --accent-foreground: oklch(0.25 0 0);
  --destructive: oklch(0.55 0.2 25);             /* red */
  --destructive-foreground: oklch(0.98 0 0);
  --success: oklch(0.6 0.18 145);                /* green */
  --warning: oklch(0.7 0.17 70);                 /* amber */
  --border: oklch(0.88 0.005 260);
  --input: oklch(0.88 0.005 260);
  --ring: oklch(0.55 0.2 260);
  --radius: 0.625rem;
  --sidebar: oklch(0.97 0.005 260);
  --sidebar-foreground: oklch(0.25 0 0);
}

.dark {
  --background: oklch(0.12 0.01 260);            /* deep dark blue-gray */
  --foreground: oklch(0.93 0 0);
  --card: oklch(0.16 0.01 260);
  --card-foreground: oklch(0.93 0 0);
  --popover: oklch(0.16 0.01 260);
  --popover-foreground: oklch(0.93 0 0);
  --primary: oklch(0.65 0.2 260);                /* brighter blue-violet in dark */
  --primary-foreground: oklch(0.12 0 0);
  --secondary: oklch(0.22 0.015 260);
  --secondary-foreground: oklch(0.93 0 0);
  --muted: oklch(0.22 0.015 260);
  --muted-foreground: oklch(0.6 0 0);
  --accent: oklch(0.22 0.015 260);
  --accent-foreground: oklch(0.93 0 0);
  --destructive: oklch(0.55 0.2 25);
  --destructive-foreground: oklch(0.98 0 0);
  --success: oklch(0.6 0.18 145);
  --warning: oklch(0.7 0.17 70);
  --border: oklch(0.26 0.015 260);
  --input: oklch(0.26 0.015 260);
  --ring: oklch(0.65 0.2 260);
  --sidebar: oklch(0.14 0.01 260);
  --sidebar-foreground: oklch(0.93 0 0);
}
```

Theme Toggle Implementation

```
// stores/themeStore.js
import { create } from 'zustand';
import { persist } from 'zustand/middleware';

export const useThemeStore = create(
  persist(
    (set) => ({
      theme: 'dark',   // 'dark' | 'light' | 'system'
      setTheme: (theme) => {
        set({ theme });
        const root = document.documentElement;
        root.classList.remove('dark', 'light');
        if (theme === 'system') {
          const sys = window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
          root.classList.add(sys);
        } else {
          root.classList.add(theme);
        }
      },
    }),
    { name: 'agent-hub-theme' }
  )
);
```

Typography Scale

Usage	Class	Size	Weight
Page title	`text-2xl font-bold`	24px / 1.5rem	700
Section title	`text-lg font-semibold`	18px / 1.125rem	600
Card title	`text-base font-medium`	16px / 1rem	500
Body text	`text-sm`	14px / 0.875rem	400
Caption / meta	`text-xs text-muted-foreground`	12px / 0.75rem	400
Code / mono	`font-mono text-sm`	14px	400

Font stack: `Inter` (primary), system fallbacks. Monospace: `JetBrains Mono` or `Fira Code`.

Spacing & Layout

Concept	Value	Tailwind Class
Page padding	24px	`p-6`
Card padding	16px	`p-4`
Gap between cards	16px	`gap-4`
Section gap	24px	`gap-6`
Sidebar width	256px	`w-64`
Sidebar collapsed	64px	`w-16`
TopBar height	56px	`h-14`
Border radius	10px	`rounded-[var(--radius)]` or `rounded-lg`

Elevation (Shadows)

Level	Use	Tailwind
0	Flat surfaces	(no shadow)
1	Cards, inputs	`shadow-sm` (dark: `shadow-none border`)
2	Dropdowns, popovers	`shadow-md`
3	Modals, dialogs	`shadow-lg`
4	Command palette	`shadow-xl`

In dark mode, prefer `border` over `shadow` for depth (shadows are invisible on dark backgrounds).

Animation & Motion

Interaction	Duration	Easing	Tailwind
Button hover/active	150ms	ease-in-out	`transition-colors duration-150`
Sidebar expand/collapse	200ms	ease-out	`transition-all duration-200`
Modal open	200ms	ease-out	`animate-in fade-in-0 zoom-in-95`
Modal close	150ms	ease-in	`animate-out fade-out-0 zoom-out-95`
Toast slide-in	300ms	spring	(Sonner handles this)
Skeleton pulse	1.5s	ease-in-out	`animate-pulse`
Voice orb pulse	1s	ease-in-out	Custom `@keyframes pulse` with scale + opacity

Rule: **Never exceed 300ms** for UI feedback. Prefer `150ms` for micro-interactions.

---

5. UI Component Library

Core Stack

Layer	Library	Purpose
Primitives	**shadcn/ui** (Radix UI based)	Button, Input, Dialog, Select, Tabs, Tooltip, etc.
Toast	**Sonner** (`sonner`)	Notification toasts — success, error, info, promise
Charts	**Recharts**	GenUI line/bar/pie charts
Icons	**Lucide React**	Consistent 24x24 icon set, tree-shakeable
Markdown	**react-markdown** + `remark-gfm`	Render agent markdown responses
Code Syntax	**Shiki** or **highlight.js**	CodeBlock syntax highlighting
Command Palette	**cmdk** (shadcn/ui wraps this)	⌘K global command palette
Date	**date-fns**	Lightweight date formatting (no moment.js)

shadcn/ui Setup

```
# Initialize shadcn/ui in the dashboard
npx shadcn@latest init

# Add components as needed (copy-paste, not npm dep)
npx shadcn@latest add button input card dialog dropdown-menu
npx shadcn@latest add tabs tooltip badge avatar select switch
npx shadcn@latest add slider skeleton scroll-area separator sheet
npx shadcn@latest add popover command sonner
```

Toast Usage Pattern (Sonner)

```
import { toast } from 'sonner';

// Success
toast.success('Persona created', { description: 'Nova is ready to chat' });

// Error
toast.error('Connection failed', { description: 'Check your internet' });

// Info
toast.info('Plugin enabled', { description: 'Brave Search is now active' });

// Promise (loading → success/error)
toast.promise(enablePlugin(pluginId), {
  loading: 'Enabling plugin...',
  success: 'Plugin enabled!',
  error: 'Failed to enable plugin',
});

// Action toast
toast('Session expired', {
  action: { label: 'Reconnect', onClick: () => reconnect() },
});
```

Loading States

Every async operation follows this pattern:

State	UI
Loading	`<Skeleton />` for content, spinner for actions
Empty	`<EmptyState icon={…} title="…" description="…" action={…} />`
Error	Sonner toast + inline error message if critical
Success	Sonner toast (non-blocking) or inline confirmation

```
// Consistent pattern for all data-fetching components
function PersonaList() {
  const { personas, loading, error } = usePersonaStore();

  if (loading) return <PersonaListSkeleton />;
  if (error) return <EmptyState icon={AlertCircle} title="Failed to load" action={{ label: 'Retry', onClick: refetch }} />;
  if (!personas.length) return <EmptyState icon={Users} title="No personas yet" description="Create your first AI persona" action={{ la

  return <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">{personas.map(p => <PersonaCard key={p.id} persona={p} />)}</div>;
}
```

6. Page-by-Page UI/UX Specs

6.1 Login Page (`/login`)

```
┌──────────────────────────────────────────────┐
│                                              │
│              ┌──────────────────┐            │
│              │   Agent Hub      │            │
│              │                  │            │
│              │ Your AI agent    │            │
│              │ across all devices│           │
│              │                  │            │
│              │ ┌──────────────┐ │            │
│              │ │ G  Sign in with│ │          │
│              │ │    Google    │ │            │
│              │ └──────────────┘ │            │
│              │                  │            │
│              │    or email      │            │
│              │ ┌──────────────┐ │            │
│              │ │ email@...    │ │            │
│              │ └──────────────┘ │            │
│              │ ┌──────────────┐ │            │
│              │ │   Continue →  │ │            │
│              │ └──────────────┘ │            │
│              └──────────────────┘            │
│                                              │
│  Background: subtle gradient dark            │
└──────────────────────────────────────────────┘
```

Element	Spec
Layout	Centered card on gradient background
Logo	SVG orb icon + “Agent Hub” text
Primary CTA	Google Sign-In button (Firebase Auth)
Secondary	Email/password (Firebase Auth `signInWithEmailAndPassword`)
After login	Redirect to `/` (DashboardPage)
Theme	Always dark on login page (brand consistency)

6.2 Dashboard Page (`/`) — Main Workspace

This is the **primary screen** users spend 90% of their time on.

```
┌──────────────────────────────────────────────────────────────────────┐
│ ┌──────┐ Agent Hub                    🔌 3 clients  🌙  👤 User ▾│  ← TopBar
│ │  ≡   │                                                          │
│ ├──────┼──────────────────────────────────────────────────────────┤
│ │ 🏠 D │  ┌─ Active Persona ──────────────────────────────┐       │
│ │ 👥 P │  │ ● Nova (Analyst) · Charon voice · 3 plugins    │       │
│ │ 🧩 M │  └────────────────────────────────────────────────┘       │
│ │ 📋 S │                                                          │
│ │ 🔌 C │  ┌─ Chat + GenUI Area (scrollable) ──────────────┐       │
│ │ ⚙️ S │  │                                                │       │
│ │      │  │  You: "Show me Tesla's stock this year"        │       │
│ │ ──── │  │                                                │       │
│ │      │  │  Nova: "Here's Tesla's YTD performance..."     │       │
│ │ Past │  │  ┌──────────────────────────────┐              │       │
│ │ Sess │  │  │  📈  Tesla (TSLA) YTD        │              │       │
│ │ ions │  │  │  [=====LINE CHART AREA=====]  │              │       │
│ │      │  │  │  +15% YTD · Feb dip · Recovery │              │       │
│ │ s001 │  │  └──────────────────────────────┘              │       │
│ │ s002 │  │                                                │       │
│ │ s003 │  │  You: "Compare with Ford"                      │       │
│ │      │  │                                                │       │
│ │      │  │  Nova: "Here's the comparison table..."        │       │
│ │      │  │  ┌──────────────────────────────┐              │       │
│ │      │  │  │  TSLA vs F — 2026 Financials  │              │       │
│ │      │  │  │  ┌──────┬────────┬─────────┐  │              │       │
│ │      │  │  │  │      │  TSLA  │    F    │  │              │       │
│ │      │  │  │  │ Rev  │ $96.7B │ $176B   │  │              │       │
│ │      │  │  │  │ P/E  │  52.3  │  11.2   │  │              │       │
│ │      │  │  │  └──────┴────────┴─────────┘  │              │       │
│ │      │  │  └──────────────────────────────┘              │       │
│ │      │  │                                                │       │
│ │      │  └────────────────────────────────────────────────┘       │
│ │      │                                                          │
│ │      │  ┌─ Input Area ──────────────────────────────────┐       │
│ │      │  │                                                │       │
│ │      │  │  ┌──────────────────┐  ┌──┐  ┌──────────┐     │       │
│ │      │  │  │ Ask anything...  │  │🎤│  │ 🎙 Voice │     │       │
│ │      │  │  └──────────────────┘  └──┘  └──────────┘     │       │
│ │      │  │                                                │       │
│ │      │  │        ┌──────────┐                            │       │
│ │      │  │        │ ◉ (Orb)  │    ← Voice Orb             │       │
│ │      │  │        │Listening..│                           │       │
│ │      │  │        └──────────┘                            │       │
│ │      │  └────────────────────────────────────────────────┘       │
│ └──────┘                                                          │
└──────────────────────────────────────────────────────────────────────┘
```

Dashboard Layout Breakdown

Zone	Component	Behavior
TopBar	`<TopBar />`	Logo, connected clients count, theme toggle, user avatar/menu
Sidebar	`<Sidebar />`	Navigation icons + labels, collapsible, past sessions list at bottom
Active Persona	`<PersonaBanner />`	Shows current persona name, voice, active plugin count. Click to switch
Chat Area	`<ChatPanel />`	Scrollable message list. Each message is `<MessageBubble />` which may contain text OR GenUI components
GenUI Blocks	`<GenUIRenderer />`	Renders inline within chat: charts, tables, code blocks, images, etc.
Input Area	`<ChatInput />`	Text input + attachment button + voice toggle button
Voice Orb	`<VoiceOrb />`	Pulsing orb when voice is active. States: idle → listening → processing → speaking
Session List	`<SessionList />`	Bottom section of sidebar, shows recent session titles

Voice Orb States

State	Visual	Description
Idle	Dim orb, static	Voice not active
Listening	Blue glow, gentle pulse, waveform	Mic active, capturing audio
Processing	Purple glow, spinning ring	Agent thinking
Speaking	Blue-violet glow, waveform animates to output	Agent audio playing
Error	Red glow, shake	Connection lost or error

```jsx
// VoiceOrb.jsx states
const orbStyles = {
  idle:       'bg-muted scale-100',
  listening:  'bg-primary/20 scale-110 animate-pulse ring-2 ring-primary/50',
  processing: 'bg-primary/30 scale-105 animate-spin',
  speaking:   'bg-primary/40 scale-110',
  error:      'bg-destructive/20 scale-100 animate-shake',
};
```

6.3 Personas Page (`/personas`)

```
┌──────────────────────────────────────────────────┐
│ Sidebar │  Personas                       [+ Create]    │
│         │                                               │
│         │  ┌──────────┐ ┌──────────┐ ┌──────────┐ │
│         │  │ 🟣 Nova  │ │ 🔵 Atlas │ │ 🟢 Sage  │ │
│         │  │ Analyst  │ │ Coder    │ │ Researcher │ │
│         │  │ Charon   │ │ Kore     │ │ Aoede    │ │
│         │  │ 3 plugins │ │ 5 plugins │ │ 4 plugins │ │
│         │  │          │ │          │ │          │ │
│         │  │ [Activate] │ │ ● Active │ │ [Activate] │ │
│         │  └──────────┘ └──────────┘ └──────────┘ │
│         │                                               │
│         │  ┌──────────┐ ┌──────────┐                 │
│         │  │ 🟡 Spark │ │ 🔴 Claire │                 │
│         │  │ Creative │ │ Assistant │                 │
│         │  │ Leda     │ │ Puck     │                 │
│         │  │ 2 plugins │ │ 6 plugins │                 │
│         │  │          │ │          │                 │
│         │  │ [Activate] │ │ [Activate] │                 │
│         │  └──────────┘ └──────────┘                 │
└──────────────────────────────────────────────────┘
```

Element	Spec
Layout	Responsive grid: 1 col (mobile), 2 col (tablet), 3 col (desktop)
Card	Color-coded dot, persona name, role, voice name, plugin count, activate/active button
Create	Opens `<PersonaEditor />` sheet from right
Edit	Click card → opens editor with pre-filled values
Delete	Inside editor → destructive button with `<ConfirmDialog />`
Active indicator	Green dot + "Active" label; only one active at a time

Persona Editor (Sheet)

Field	Input Type	Description
Name	Text input	Persona display name
Role	Text input	e.g., "Financial Analyst", "Code Expert"
Color	Color picker (6 preset colors)	Card accent
Voice	Select dropdown	Puck, Charon, Kore, Fenrir, Aoede, Leda, Orus, Zephyr
System Instruction	Textarea	Agent instruction prompt
Greeting	Text input	What the agent says on session start
Plugins	Multi-select checkboxes	Which MCPs are enabled for this persona
Preview	`<VoicePreview />` button	Play a 3-second voice sample

6.4 Plugin Store (`/plugins`)

```
┌──────────────────────────────────────────────────────────┐
│ Sidebar │  Plugin Store                  🔍 [Search...]     │
│         │                                                   │
│         │  [All] [Productivity] [Finance] [Dev] [Health] ... │ ← Category tabs
│         │                                                   │
│         │  ┌──────────────┐ ┌──────────────┐ ┌──────────┐ │
│         │  │ 🔍 Brave     │ │ 📊 Financial  │ │ 🐙 GitHub│ │
│         │  │    Search    │ │    Datasets   │ │          │ │
│         │  │              │ │               │ │          │ │
│         │  │ Web search   │ │ Stock data,   │ │ Issues,  │ │
│         │  │ Privacy-first│ │ earnings      │ │ PRs, etc │ │
│         │  │              │ │               │ │          │ │
│         │  │ ◉ Enabled    │ │ ○ Enable      │ │ ○ Enable │ │
│         │  └──────────────┘ └──────────────┘ └──────────┘ │
│         │                                                   │
│         │  ... (more cards)                                 │
└──────────────────────────────────────────────────────────┘
```

Element	Spec
Layout	Grid of cards with category tab filter + search
Card	Icon, name, short description, enable/disable toggle
categories	All, Productivity, Finance, Developer, Health, Education, Marketing, Communication
Toggle	shadcn `<Switch />` with toast on toggle
Detail	Click card → `<MCPDetail />` sheet with full description, config fields (API keys), connection status
Config	Some MCPs require API keys → secure input field in detail sheet, saved to Firestore (encrypted)

6.5 Sessions Page (`/sessions`)

Element	Spec
Layout	List view with search + date filter
Item	Session title (auto-generated from first message), persona used, date, message count, duration
Click	Opens session in chat view (read-only replay)
Delete	Swipe or hover → delete icon → `<ConfirmDialog />`
Search	Full-text search across session transcripts

6.6 Clients Page (`/clients`)

```
┌──────────────────────────────────────────────────────────┐
│ Sidebar │  Connected Devices                              │
│         │                                                 │
│         │  ┌──────────────────────────────────────────┐   │
│         │  │  🌐 Web Dashboard   · Chrome · macOS      │   │
│         │  │  ● Online · Connected 2h ago · This device │   │
│         │  └──────────────────────────────────────────┘   │
│         │                                                 │
│         │  ┌──────────────────────────────────────────┐   │
│         │  │  📱 Mobile PWA      · Safari · iOS 18     │   │
│         │  │  ● Online · Connected 15m ago             │   │
│         │  └──────────────────────────────────────────┘   │
│         │                                                 │
│         │  ┌──────────────────────────────────────────┐   │
│         │  │  🖥️ Desktop Client   · Windows 11         │   │
│         │  │  ○ Offline · Last seen 3h ago             │   │
│         │  └──────────────────────────────────────────┘   │
│         │                                                 │
│         │  ┌──────────────────────────────────────────┐   │
│         │  │  🧩 Chrome Extension · Chrome v132        │   │
│         │  │  ● Online · Connected 5m ago              │   │
│         │  └──────────────────────────────────────────┘   │
│         │                                                 │
│         │  ┌──────────────────────────────────────────┐   │
│         │  │  👓 ESP32 Glasses    · WiFi               │   │
│         │  │  ○ Offline · Never connected              │   │
│         │  └──────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

6.7 Settings Page (`/settings`)

Section	Fields
Profile	Display name, avatar
Theme	Dark / Light / System toggle (3-way radio)
Audio	Input device select, output device select, voice activity sensitivity slider
Notifications	Desktop notifications toggle, sound toggle
API Keys	Personal API keys for MCPs that require them (masked input, eye toggle)
Data	Export sessions, delete all data (destructive)
About	Version, GitHub link, hackathon info

---

7. API Layer & Security

HTTP API Client

A single `api.js` wrapper handles all HTTP requests with automatic token injection, error handling, and toast notifications.

```
// lib/api.js
import { useAuthStore } from '@/stores/authStore';
import { toast } from 'sonner';

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000';

async function request(path, options = {}) {
  const token = useAuthStore.getState().token;

  const res = await fetch(`${API_BASE}${path}`, {
    ...options,
    headers: {
      'Content-Type': 'application/json',
      ...(token && { Authorization: `Bearer ${token}` }),
      ...options.headers,
    },
  });

  if (res.status === 401) {
    useAuthStore.getState().logout();
    toast.error('Session expired', { description: 'Please sign in again' });
    throw new Error('Unauthorized');
  }

  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    const msg = body.detail || `Request failed (${res.status})`;
    toast.error('Error', { description: msg });
    throw new Error(msg);
  }

  return res.json();
}

export const api = {
  get:    (path)        => request(path),
  post:   (path, data)  => request(path, { method: 'POST', body: JSON.stringify(data) }),
  put:    (path, data)  => request(path, { method: 'PUT', body: JSON.stringify(data) }),
  patch:  (path, data)  => request(path, { method: 'PATCH', body: JSON.stringify(data) }),
  delete: (path)        => request(path, { method: 'DELETE' }),
};
```

REST API Endpoints

Method	Path	Description	Auth
`POST`	`/auth/verify`	Verify Firebase ID token, return user profile	No (this establishes auth)
`GET`	`/personas`	List user's personas	✅
`POST`	`/personas`	Create new persona	✅
`PUT`	`/personas/:id`	Update persona	✅
`DELETE`	`/personas/:id`	Delete persona	✅
`GET`	`/mcp/catalog`	List available MCP plugins	✅
`GET`	`/mcp/installed`	List user's enabled plugins	✅
`POST`	`/mcp/toggle`	Enable/disable a plugin	✅
`PUT`	`/mcp/:id/config`	Update plugin config (API keys)	✅
`GET`	`/sessions`	List user's sessions	✅
`GET`	`/sessions/:id`	Get session detail + messages	✅
`DELETE`	`/sessions/:id`	Delete session	✅
`GET`	`/clients`	List connected devices	✅
`GET`	`/health`	Server health check	No

WebSocket Endpoints

Path	Purpose	Auth
`wss://host/ws/live/{session_id}`	Bidi audio streaming (ADK `run_live()`)	Token in first message
`wss://host/ws/events/{session_id}`	Dashboard event stream (GenUI, status, transcription)	Token in first message

Security Implementation

1. Authentication (Firebase Auth → Backend Verification)

```
Client                        Backend
  |                              |
  |  Firebase signIn() ──────►   |
  |  ◄──────── ID Token          |
  |                              |
  |  GET /personas               |
  |  Authorization: Bearer <token>
  |  ──────────────────────────► |
  |                              |  firebase_admin.auth.verify_id_token(token)
  |                              |  → Extract uid, email
  |                              |
  |  ◄──────── 200 + data        |
```

```python
# backend/app/middleware/auth_middleware.py
from firebase_admin import auth as firebase_auth

async def verify_firebase_token(token: str) -> dict:
    """Verify Firebase ID token. Returns decoded claims or raises."""
    decoded = firebase_auth.verify_id_token(token)
    return {"uid": decoded["uid"], "email": decoded.get("email")}
```

2. WebSocket Auth (Token in First Message)

```javascript
// Client sends token as first message after WS connect
const ws = new WebSocket(`wss://${host}/ws/live/${sessionId}`);
ws.onopen = () => {
  ws.send(JSON.stringify({ type: 'auth', token: firebaseIdToken }));
};
```

```python
# Backend validates first message
@app.websocket("/ws/live/{session_id}")
async def ws_live(websocket: WebSocket, session_id: str):
    await websocket.accept()
    first_msg = await websocket.receive_json()
    if first_msg.get("type") != "auth":
        await websocket.close(code=4001, reason="Auth required")
        return
    try:
        user = await verify_firebase_token(first_msg["token"])
    except Exception:
        await websocket.close(code=4003, reason="Invalid token")
        return
    # Proceed with authenticated session...
```

3. Input Sanitization

- All user text input sanitized before sending to Gemini (strip HTML, limit length)
- Persona system instructions are sanitized on the backend (prevent prompt injection via persona config)
- MCP API keys encrypted at rest in Firestore using Google KMS

4. CORS Configuration

```
# Only allow our frontend origins
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173",         # Vite dev
        "https://agent-hub.web.app",     # Production
    ],
    allow_methods=["*"],
    allow_headers=["*"],
    allow_credentials=True,
)
```

5. Rate Limiting

```
# Middleware: per-user rate limits
RATE_LIMITS = {
    "POST /personas": "10/minute",
    "POST /mcp/toggle": "30/minute",
    "WS /ws/live": "5 concurrent",
}
```

6. Security Headers

```
@app.middleware("http")
async def security_headers(request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    response.headers["Permissions-Policy"] = "camera=(), microphone=(self)"
    return response
```

---

8. WebSocket Protocol

Message Types (Client → Server)

```
// Authentication (first message after connect)
{ "type": "auth", "token": "<firebase_id_token>" }

// Audio data (binary frame — NOT JSON)
// Raw PCM 16-bit, 16kHz, mono — sent as binary WebSocket frame

// Text message
{ "type": "text", "content": "What's on my schedule?" }

// Image (camera frame)
{ "type": "image", "data": "<base64_jpeg>", "mimeType": "image/jpeg" }

// Control
{ "type": "control", "action": "switch_persona", "personaId": "nova-123" }
{ "type": "control", "action": "start_voice" }
{ "type": "control", "action": "stop_voice" }
```

Message Types (Server → Client)

```
// Audio response (binary frame — PCM 24kHz)

// Text/GenUI response
{
  "type": "response",
  "content": "Here's Tesla's performance...",
  "genui": {                              // Optional — triggers GenUI rendering
    "type": "chart",
    "chartType": "line",
    "title": "Tesla (TSLA) YTD",
    "data": [...]
  }
}

// Transcription updates
{ "type": "transcription", "direction": "input", "text": "What's on my", "finished": false }
{ "type": "transcription", "direction": "input", "text": "What's on my schedule?", "finished": true }
{ "type": "transcription", "direction": "output", "text": "You have 3 meetings", "finished": true }

// Agent status
{ "type": "status", "state": "listening" }      // idle, listening, processing, speaking, error
{ "type": "status", "state": "processing", "detail": "Calling Brave Search..." }

// Tool execution
{ "type": "tool_start", "tool": "brave_search", "query": "TSLA stock 2026" }
{ "type": "tool_end", "tool": "brave_search", "success": true }

// Cross-client event
{ "type": "cross_client", "action": "note_saved", "target": "web", "data": {...} }

// Persona switched
{ "type": "persona_changed", "persona": { "id": "...", "name": "Nova", "voice": "Charon" } }

// Connection events
{ "type": "connected", "sessionId": "...", "resumedFrom": "..." }
{ "type": "error", "code": "RATE_LIMITED", "message": "Too many requests" }
```

WebSocket Hook

```
// hooks/useWebSocket.js
import { useRef, useEffect, useCallback } from 'react';
import { useAuthStore } from '@/stores/authStore';
import { useChatStore } from '@/stores/chatStore';
import { toast } from 'sonner';

const WS_URL = import.meta.env.VITE_WS_URL || 'ws://localhost:8000';
const RECONNECT_DELAYS = [1000, 2000, 4000, 8000, 16000]; // Exponential backoff

export function useWebSocket(sessionId) {
  const wsRef = useRef(null);
  const reconnectAttempt = useRef(0);
  const token = useAuthStore((s) => s.token);
  const addMessage = useChatStore((s) => s.addMessage);
  const setAgentState = useChatStore((s) => s.setAgentState);

  const connect = useCallback(() => {
    if (!sessionId || !token) return;

    const ws = new WebSocket(`${WS_URL}/ws/live/${sessionId}`);
    wsRef.current = ws;

    ws.onopen = () => {
      ws.send(JSON.stringify({ type: 'auth', token }));
      reconnectAttempt.current = 0;
      toast.success('Connected');
    };

    ws.onmessage = (event) => {
      if (event.data instanceof Blob) {
        // Binary audio data → playback
        useChatStore.getState().enqueueAudio(event.data);
        return;
      }
      const msg = JSON.parse(event.data);
      switch (msg.type) {
        case 'response':       addMessage(msg); break;
        case 'transcription':  useChatStore.getState().updateTranscript(msg); break;
        case 'status':         setAgentState(msg.state); break;
        case 'tool_start':     useChatStore.getState().setToolActive(msg.tool, true); break;
        case 'tool_end':       useChatStore.getState().setToolActive(msg.tool, false); break;
        case 'error':          toast.error(msg.message); break;
      }
    };

    ws.onclose = (e) => {
      if (e.code === 4003) { toast.error('Auth failed'); return; }
      const delay = RECONNECT_DELAYS[Math.min(reconnectAttempt.current, RECONNECT_DELAYS.length - 1)];
      reconnectAttempt.current++;
      toast.info('Reconnecting...', { description: `Attempt ${reconnectAttempt.current}` });
      setTimeout(connect, delay);
    };

    ws.onerror = () => toast.error('Connection error');
  }, [sessionId, token]);

  useEffect(() => { connect(); return () => wsRef.current?.close(); }, [connect]);

  const sendText = (text) => wsRef.current?.send(JSON.stringify({ type: 'text', content: text }));
  const sendAudio = (pcmData) => wsRef.current?.send(pcmData); // Binary
  const sendImage = (base64) => wsRef.current?.send(JSON.stringify({ type: 'image', data: base64, mimeType: 'image/jpeg' }));
  const sendControl = (action, data = {}) => wsRef.current?.send(JSON.stringify({ type: 'control', action, ...data }));

  return { sendText, sendAudio, sendImage, sendControl, ws: wsRef };
}
```

9. State Management

Store Architecture (Zustand)

Each domain gets its own store. Stores are independent — no cross-store imports (use subscriptions if needed).

```
stores/
├── authStore.js      # user, token, login(), logout()
├── chatStore.js      # messages[], transcript, agentState, voice state, audio queue
├── personaStore.js   # personas[], activePersona, CRUD actions
├── mcpStore.js       # catalog[], installed[], toggle()
├── clientStore.js    # clients[], refresh()
├── sessionStore.js   # sessions[], activeSessionId
├── themeStore.js     # theme ('dark'|'light'|'system'), setTheme()
└── uiStore.js        # sidebarOpen, commandPaletteOpen, activeModal
```

Store Pattern

```jsx
// stores/personaStore.js
import { create } from 'zustand';
import { api } from '@/lib/api';
import { toast } from 'sonner';

export const usePersonaStore = create((set, get) => ({
  personas: [],
  activePersona: null,
  loading: false,
  error: null,

  fetchPersonas: async () => {
    set({ loading: true, error: null });
    try {
      const data = await api.get('/personas');
      set({ personas: data, loading: false });
    } catch (err) {
      set({ error: err.message, loading: false });
    }
  },

  createPersona: async (persona) => {
    const data = await api.post('/personas', persona);
    set((s) => ({ personas: [...s.personas, data] }));
    toast.success('Persona created', { description: data.name });
    return data;
  },

  activatePersona: (persona) => {
    set({ activePersona: persona });
    toast.info(`Switched to ${persona.name}`);
  },

  deletePersona: async (id) => {
    await api.delete(`/personas/${id}`);
    set((s) => ({
      personas: s.personas.filter((p) => p.id !== id),
      activePersona: s.activePersona?.id === id ? null : s.activePersona,
    }));
    toast.success('Persona deleted');
  },
}));
```

10. Accessibility & Responsive Design

Responsive Breakpoints

Breakpoint	Width	Layout
Mobile	< 640px (`sm:`)	Bottom nav, full-width chat, no sidebar
Tablet	640-1024px (`md:`)	Collapsible sidebar (icons only), chat + sidebar
Desktop	> 1024px (`lg:`)	Full sidebar + chat + optional right panel

Mobile Layout

```
┌──────────────────────┐
│  Agent Hub   🌙  👤  │  ← Slim TopBar
├──────────────────────┤
│                      │
│  Chat messages       │  ← Full width
│  + GenUI blocks      │
│                      │
│                      │
├──────────────────────┤
│ [Type a message... 🎙] │  ← Input bar
├──────────────────────┤
│ 🏠  👥  🧩  📋  ⚙️  │  ← Bottom nav (MobileNav)
└──────────────────────┘
```

Accessibility Checklist

Requirement	Implementation
Keyboard navigation	All interactive elements focusable; Tab order logical; Escape closes modals
Screen reader	ARIA labels on icons, live regions for transcription, role="alert" for toasts
Focus visible	`focus-visible:ring-2 focus-visible:ring-ring` on all interactive elements
Color contrast	WCAG AA minimum (4.5:1 text, 3:1 large text) — both light and dark modes
Reduced motion	`motion-reduce:animate-none` on all animations
Voice Orb	aria-label announces state ("Listening", "Agent speaking")
Touch targets	Minimum 44x44px on mobile

11. Performance Guidelines

Bundle Optimization

Strategy	Implementation
Code splitting	React.lazy() for each page: `const PersonasPage = lazy(() => import('./pages/PersonasPage'))`
Tree shaking	Import specific icons: `import { Mic } from 'lucide-react'` (not `import * as icons`)
Image optimization	WebP format, lazy loading, width/height attributes
Font loading	`font-display: swap` for Inter + JetBrains Mono
Chunk analysis	`vite-plugin-visualizer` to monitor bundle size

Target Metrics

Metric	Target
First Contentful Paint	< 1.5s
Largest Contentful Paint	< 2.5s
Total JS bundle	< 200KB gzipped (initial)
WebSocket latency	< 100ms (audio round-trip)

Audio Performance

Concern	Solution
Audio capture glitches	AudioWorklet (not ScriptProcessorNode) — runs on separate thread
Playback gaps	Ring buffer (180s capacity) — absorbs network jitter
Memory leaks	Clean up AudioContext on component unmount
Mobile battery	Stop AudioContext when voice is deactivated

---

12. Developer Workflow

Local Development

```
# Terminal 1 — Backend
cd backend
uv run uvicorn app.main:app --reload --port 8000

# Terminal 2 — Frontend
cd dashboard
npm run dev    # Vite at http://localhost:5173

# Terminal 3 — (Optional) Desktop Client
cd desktop-client
uv run python src/main.py
```

Environment Variables

```
# backend/.env.example
GOOGLE_API_KEY=                      # Gemini API key
GOOGLE_CLOUD_PROJECT=                # GCP project ID
E2B_API_KEY=                         # E2B sandbox key
FIREBASE_SERVICE_ACCOUNT=            # Path to Firebase service account JSON

# dashboard/.env.example
VITE_API_URL=http://localhost:8000   # Backend URL
VITE_WS_URL=ws://localhost:8000     # WebSocket URL
VITE_FIREBASE_API_KEY=               # Firebase web config
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
```

Code Quality (Pre-Commit)

```
# Backend
uv run ruff check app/                # Lint
uv run ruff format app/               # Format

# Frontend
npx eslint src/ --fix                 # Lint
npx prettier --write src/             # Format
```

Component Development Pattern

When building a new component:

1. **Create component file** in the right folder (`components/persona/PersonaCard.jsx`)
2. **Accept props** — data in, callbacks out. No internal fetch calls in basic components
3. **Use shadcn/ui primitives** — `<Card>`, `<Badge>`, `<Button>`, not raw HTML
4. **Use Tailwind** only — no CSS modules, no styled-components, no inline styles
5. **Apply dark mode** via CSS variables — never hardcode colors
6. **Handle loading/empty/error** — use `<Skeleton />`, `<EmptyState />`, toast
7. **Use `cn()` helper** for conditional classes: `cn('base-class', active && 'active-class')`

```
// lib/cn.js
import { clsx } from 'clsx';
import { twMerge } from 'tailwind-merge';

export function cn(...inputs) {
  return twMerge(clsx(inputs));
}
```

Commit Before You Push Checklist

- [ ] `ruff check` and `eslint` pass with 0 errors
- [ ] No `console.log` in committed code (use structured logging)
- [ ] No hardcoded API keys, URLs, or secrets
- [ ] Dark mode tested (toggle and verify)
- [ ] Mobile responsive tested (Chrome DevTools device mode)
- [ ] Loading and error states handled
- [ ] Toast used for async feedback (not alerts)

Quick Reference Card

Import Convention

```
// 1. React / libraries
import { useState, useEffect } from 'react';
import { toast } from 'sonner';

// 2. UI primitives
import { Button } from '@/components/ui/button';
import { Card, CardHeader, CardTitle, CardContent } from '@/components/ui/card';

// 3. App components
import { PersonaCard } from '@/components/persona/PersonaCard';

// 4. Stores / hooks
import { usePersonaStore } from '@/stores/personaStore';
import { useAuth } from '@/hooks/useAuth';

// 5. Utils
import { cn } from '@/lib/cn';
import { api } from '@/lib/api';
```

Naming Convention

Thing	Convention	Example
Component file	PascalCase.jsx	`PersonaCard.jsx`
Hook file	camelCase.js	`useWebSocket.js`
Store file	camelCase.js	`personaStore.js`
Utility file	camelCase.js	`formatters.js`
CSS variable	kebab-case	`--primary-foreground`
Zustand store	`use[Name]Store`	`usePersonaStore`
Hook	`use[Name]`	`useWebSocket`
API function	`verb + Noun`	`fetchPersonas`, `createPersona`
Event handler	`handle + Event`	`handleSubmit`, `handleToggle`
Boolean prop	`is/has/can`	`isActive`, `hasError`, `canDelete`